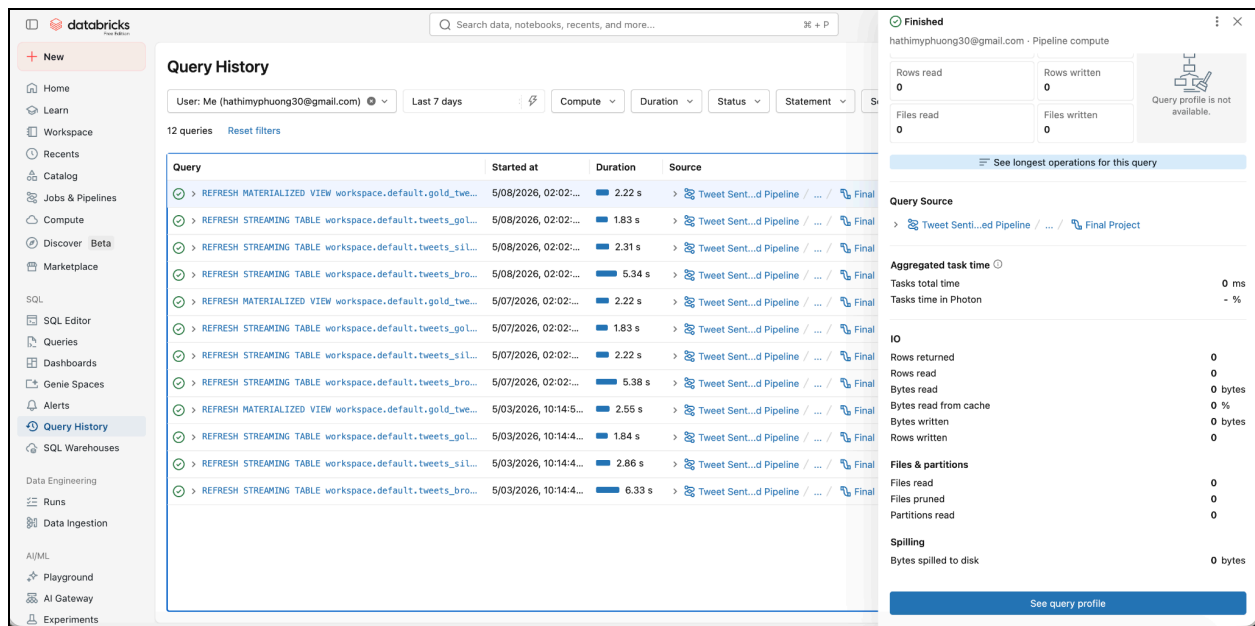




Because the dataset contains only about 42,000 tweets, it does not place significant pressure on memory, compute, or storage resources. As a result, common large-scale performance issues such as spill, skew, and heavy shuffle are not meaningfully observable in this workload.

Execution metrics show that zero bytes were spilled to disk, indicating that all operations were completed entirely in memory, which is expected given the small dataset size. The primary source of shuffle in this pipeline comes from two wide transformations: the explode on the mentions array in the Silver layer and the groupBy on the mention column in the Gold Aggregation layer, both of which require data to be redistributed across partitions. The groupBy is also the most memory-intensive step, as Spark must maintain intermediate aggregation state during the shuffle. At the current dataset size, neither the shuffle overhead nor the memory pressure poses any issue. At scale, however, these same operations could exhaust executor memory and trigger spill to disk, while shuffle costs would grow significantly. To mitigate this, unnecessary wide transformations should be minimized, the number of shuffle partitions should be increased to distribute load more evenly, and data should be repartitioned by the mention column to reduce skew. The optimal partition count typically depends on available CPU cores. Caching intermediate results that are reused downstream and applying early filtering before wide transformations can further reduce both shuffle volume and spill risk.

In terms of storage, the pipeline uses Delta tables for each stage, which provides efficient data management and query performance. Given the small dataset size, the number of files and overall storage are minimal, and no performance issues are observed. However, as data volume grows, Delta tables can suffer from the small files problem, where many tiny files degrade read performance and increase metadata overhead. To address this, the OPTIMIZE command can be

used to compact small files into larger ones, improving read efficiency. Additionally, ZORDER indexing on frequently queried columns such as mention can improve query performance by colocating related data, reducing scan time for aggregation queries.

The most significant performance consideration in this pipeline is serialization overhead introduced by the use of a Python UDF for sentiment inference. The `sentiment_udf` requires data to be transferred between Spark's JVM and the Python runtime, which incurs serialization and deserialization costs. This overhead exists regardless of dataset size and can become a major bottleneck at scale, particularly for compute-intensive operations like model inference. To improve performance, the Python UDF could be replaced with more efficient alternatives such as Pandas UDFs, which leverage vectorized operations, or by using Spark-native functions and MLflow model serving to avoid repeated cross-language data transfer.

Taken together, the pipeline performs well for its current workload, but several design choices would become bottlenecks at greater scale. Proactively addressing spill risk, shuffle cost, file proliferation, and serialization overhead will ensure the system remains performant as data volume and complexity grow.